

ONLINE LEARNING DATABASE

Schema & Reporting

Database design, sample data and ten SQL reports: the code, the results, the method and the functions explained.

1

students

who is learning

2

courses

what is taught

3

enrollments

who took which course

4

assessments

quiz, assignment and exam scores

Agenda

Schema, sample data and SQL reports for the Online Learning database.

01

Creation of Database

The database and its character set

02

Creation of Tables & Data Insertion

students, courses, enrollments, assessments

03

Report Creation & KPIs

Executive KPIs and nine SQL reports (3.1 to 3.10)

04

ERD Representation

How the four tables connect

05

Conclusion

What the schema and SQL deliver

1.1

Creation of Database

CREATE DATABASE

Create the database once, then make it the active one.

SQL CODE

```
CREATE DATABASE IF NOT EXISTS online_learning
  DEFAULT CHARACTER SET utf8mb4;
USE online_learning;
```

1

CREATE DATABASE

creates a new database; IF NOT EXISTS avoids an error on re-run.

2

DEFAULT CHARACTER SET utf8mb4

stores full Unicode text, including names from any country.

3

USE

selects the database as the active database.

02

Creation of Tables & Inserting Data

students, courses, enrollments and assessments, each with sample rows

Stores one row per learner. Email must be unique.

SQL CODE

```
CREATE TABLE students (  
  student_id INT NOT NULL,  
  full_name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL,  
  country VARCHAR(50) NOT NULL,  
  signup_date DATE NOT NULL,  
  PRIMARY KEY (student_id),  
  UNIQUE KEY uq_students_email (email)  
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;
```

Constraints explained

PRIMARY KEY	uniquely identifies each student.
UNIQUE KEY	no two students can share an email.
NOT NULL	the column must always have a value.
VARCHAR / DATE	text and calendar-date data types.

SAMPLE ROWS INSERTED

student_id	full_name	email	country	signup_date
1	Omar Garcia	omar.garcia@example.com	Germany	2025-01-13
2	Yuki Silva	yuki.silva@example.com	United States	2025-03-13
3	Grace Novak	grace.novak@example.com	Spain	2025-02-14
4	Isabella Patel	isabella.patel@example.com	United States	2025-01-16
5	Mateo Silva	mateo.silva@example.com	Spain	2025-09-16

Course catalogue: level is limited to three values and price cannot be negative.

SQL CODE

```
CREATE TABLE courses (  
  course_id      INT      NOT NULL,  
  course_title   VARCHAR(100) NOT NULL,  
  category       VARCHAR(50)  NOT NULL,  
  level          VARCHAR(20)  NOT NULL,  
  instructor     VARCHAR(100) NOT NULL,  
  price_usd      DECIMAL(8,2) NOT NULL,  
  duration_hours INT      NOT NULL,  
  PRIMARY KEY (course_id),  
  CONSTRAINT chk_courses_level CHECK  
    (level IN ('Beginner', 'Intermediate', 'Advanced')),  
  CONSTRAINT chk_courses_price CHECK (price_usd >= 0)  
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;
```

Constraints explained

DECIMAL (8 , 2)	exact numbers for price, no rounding errors.
CHECK	limits level to three values and blocks negative prices.
CONSTRAINT	gives each rule a name for clear error messages.

SAMPLE ROWS INSERTED

course_id	course_title	level	price_usd	hours
101	SQL Fundamentals	Beginner	49	12
102	Advanced SQL & Query Optim.	Advanced	99	20
103	Python for Data Analysis	Intermediate	79	24
104	Machine Learning Essentials	Advanced	129	36
105	Web Development Bootcamp	Beginner	89	40

Links students to courses: one row per enrollment, with status and progress.

SQL CODE

```
CREATE TABLE enrollments (  
  enrollment_id INT NOT NULL,  
  student_id INT NOT NULL,  
  course_id INT NOT NULL,  
  enrollment_date DATE NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  progress_pct DECIMAL(5,1) NOT NULL,  
  completion_date DATE NULL,  
  PRIMARY KEY (enrollment_id),  
  UNIQUE KEY uq_enroll_student_course (student_id, course_id),  
  KEY idx_enroll_course (course_id),  
  CONSTRAINT fk_enroll_student FOREIGN KEY (student_id)  
    REFERENCES students (student_id),  
  CONSTRAINT fk_enroll_course FOREIGN KEY (course_id)  
    REFERENCES courses (course_id),  
  CONSTRAINT chk_enroll_status CHECK  
    (status IN ('Completed', 'In Progress', 'Dropped')),  
  CONSTRAINT chk_enroll_progress CHECK  
    (progress_pct BETWEEN 0 AND 100)  
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;
```

Constraints explained

FOREIGN KEY	links each enrollment to a student and a course.
UNIQUE KEY	a student can enroll in a course only once.
CHECK	status has 3 allowed values; progress is 0 to 100.
NULL	completion_date stays empty until the course is finished.

SAMPLE ROWS INSERTED

id	student	course	enrolled	status	progress
1	43	104	2025-12-21	Dropped	51.8
2	37	104	2026-04-18	Completed	100.0
3	13	101	2026-08-17	In Progress	80.5
4	4	105	2025-03-23	Dropped	41.4
5	52	107	2026-03-14	Dropped	31.8

Quiz, assignment and final-exam attempts, each tied to one enrollment.

SQL CODE

```
CREATE TABLE assessments (  
  assessment_id INT NOT NULL,  
  enrollment_id INT NOT NULL,  
  assessment_type VARCHAR(20) NOT NULL,  
  score DECIMAL(5,1) NOT NULL,  
  max_score DECIMAL(5,1) NOT NULL,  
  attempt_date DATE NOT NULL,  
  PRIMARY KEY (assessment_id),  
  KEY idx_assess_enroll (enrollment_id),  
  CONSTRAINT fk_assess_enroll FOREIGN KEY (enrollment_id)  
    REFERENCES enrollments (enrollment_id),  
  CONSTRAINT chk_assess_type CHECK (assessment_type  
    IN ('Quiz','Assignment','Final Exam'))  
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4;
```

Constraints explained

FOREIGN KEY	every assessment belongs to an existing enrollment.
CHECK	type must be Quiz, Assignment or Final Exam.
KEY (index)	speeds up joins on enrollment_id.
score / max_score	stored separately so percentages can be calculated.

SAMPLE ROWS INSERTED

id	enrollment	type	score	max	attempt_date
1	1	Quiz	13.3	20	2026-01-05
2	1	Assignment	35.3	50	2026-01-13
3	2	Quiz	12.5	20	2026-05-04
4	2	Assignment	29.8	50	2026-05-12
5	2	Final Exam	67.2	100	2026-05-11

Reports at a glance

Ten reports, each answering one business question and each teaching one SQL technique.

3.1

Executive KPI summary

What are the headline numbers for the whole platform?

Subqueries + aggregates

3.2

Course scorecard

How does each course perform on volume, completion, dropout and score?

JOIN + weighted average

3.3

Category performance

Which course categories attract the most learners, completions and revenue?

Conditional aggregation

3.4

Monthly enrollment trend

How is enrollment growing month by month, and what is the running total?

CTE + window functions

3.5

Time to complete vs. course length

How many days do learners actually need to finish, compared with course hours?

Date arithmetic

3.6

Learner leaderboard

Who are the top 10 learners by average score (minimum 3 assessments)?

RANK() window function

3.7

Best-performing course per category

Which single course has the highest completion rate inside each category?

ROW_NUMBER() + PARTITION BY

3.8

Final-exam score distribution

How are final-exam results spread across grade bands?

CASE bucketing

3.9

Completion rate by student country

Which countries complete the most courses (at least 10 enrollments)?

HAVING threshold

3.10

Stalled learners

Which in-progress learners have gone quiet for 30+ days and are still under 90% done?

COALESCE + DATEDIFF

What are the headline numbers for the whole platform?

SQL QUERY

```
-- 3.1 Executive KPI summary
SELECT
  (SELECT COUNT(*) FROM students) AS total_students,
  (SELECT COUNT(*) FROM courses) AS total_courses,
  COUNT(*) AS total_enrollments,
  ROUND(100.0 * SUM(status = 'Completed')
        / COUNT(*), 1) AS completion_rate_pct,
  ROUND(100.0 * SUM(status = 'Dropped')
        / COUNT(*), 1) AS dropout_rate_pct,
  ROUND(100.0 * SUM(status = 'In Progress')
        / COUNT(*), 1) AS in_progress_pct,
  ROUND(AVG(progress_pct), 1) AS avg_progress_pct,
  (SELECT ROUND(100.0 * SUM(score) / SUM(max_score), 1)
   FROM assessments) AS overall_avg_score_pct
FROM enrollments;
```

RESULT

Result Grid   Filter Rows: Search Export: 							
total_stude...	total_cours...	total_enrollme...	completion_rate_...	dropout_rate_...	in_progress_...	avg_progress_...	overall_avg_score_...
60	8	190	64.2	26.3	9.5	79.4	74.6

60

Total students

8

Total courses

190

Total enrollments

64.2%

Completion rate

26.3%

Dropout rate

9.5%

In progress

79.4%

Avg progress

74.6%

Avg assessment score

Functions explained

COUNT (*)	number of enrollments (rows).	AVG ()	average progress across enrollments.
SUM(status = ...)	counts rows matching a condition.	Subquery	pulls a single value from another table.
ROUND ()	rounds a rate to 1 decimal place.	Rates	completed + dropped + in progress = 100%.

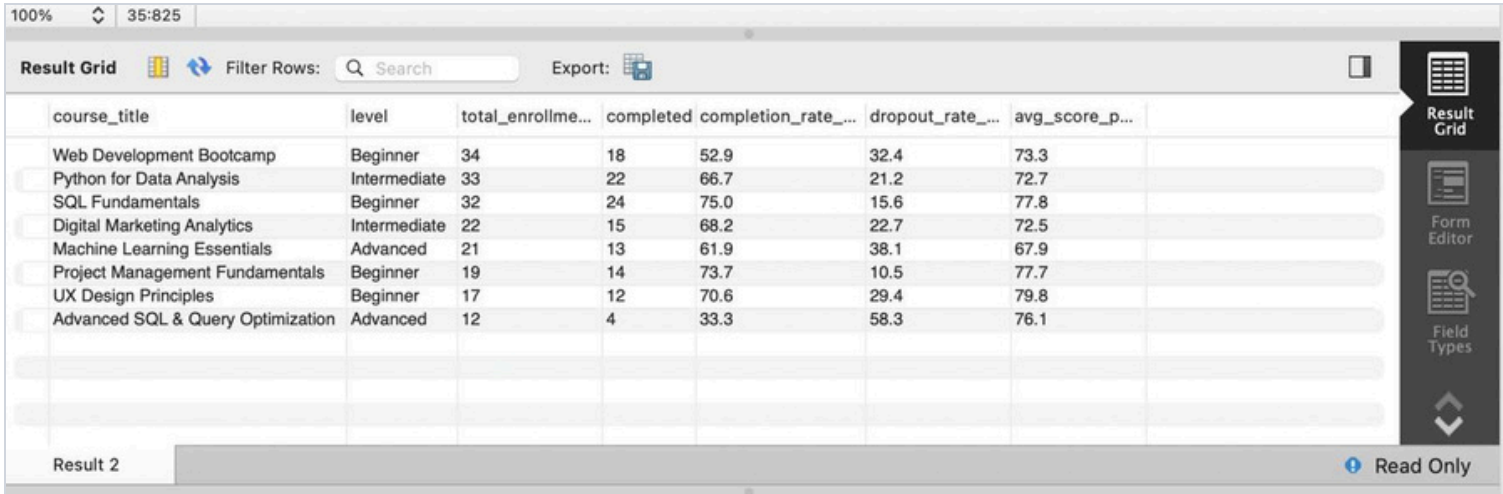
How does each course perform on volume, completion, dropout and score?

SQL QUERY

```
-- 3.2 Course scorecard: enrollments, completion and average score

SELECT
  s.course_title,
  s.level,
  s.total_enrollments,
  s.completed,
  s.completion_rate_pct,
  s.dropout_rate_pct,
  ROUND(100.0 * SUM(a.score) / SUM(a.max_score), 1) AS avg_score_pct
FROM course_scorecard s
LEFT JOIN enrollments e ON e.course_id = s.course_id
LEFT JOIN assessments a ON a.enrollment_id = e.enrollment_id
GROUP BY s.course_id, s.course_title, s.level, s.total_enrollments,
         s.completed, s.completion_rate_pct, s.dropout_rate_pct
ORDER BY s.total_enrollments DESC;
```

RESULT



course_title	level	total_enrollme...	completed	completion_rate_...	dropout_rate_...	avg_score_p...
Web Development Bootcamp	Beginner	34	18	52.9	32.4	73.3
Python for Data Analysis	Intermediate	33	22	66.7	21.2	72.7
SQL Fundamentals	Beginner	32	24	75.0	15.6	77.8
Digital Marketing Analytics	Intermediate	22	15	68.2	22.7	72.5
Machine Learning Essentials	Advanced	21	13	61.9	38.1	67.9
Project Management Fundamentals	Beginner	19	14	73.7	10.5	77.7
UX Design Principles	Beginner	17	12	70.6	29.4	79.8
Advanced SQL & Query Optimization	Advanced	12	4	33.3	58.3	76.1

KEY INSIGHT

Web Development Bootcamp is the biggest course (34 enrollments) but completes only 52.9%. SQL Fundamentals leads at 75.0%; Advanced SQL is weakest (33.3% completion, 58.3% dropout).

Method

- 1. Start from course_scorecard: one summary row per course (enrollments, completed, rates).
- 2. LEFT JOIN enrollments, then LEFT JOIN assessments, so every course keeps its rows and reaches its scores.
- 3. Average score = SUM(score) / SUM(max_score) x 100, a weighted score rather than an average of averages.
- 4. GROUP BY all non-aggregated columns and ORDER BY total_enrollments DESC (biggest course first).

Functions explained

LEFT JOIN	keeps rows even with no match on the right
SUM()	adds up scores and max scores per course
ROUND(x, 1)	rounds the result to 1 decimal place
GROUP BY	one output row per course
ORDER BY ... DESC	sorts largest to smallest

Which course categories attract the most learners, completions and revenue?

SQL QUERY

```
-- 3.3 Category performance

SELECT
  c.category,
  COUNT(DISTINCT c.course_id)                AS courses,
  COUNT(e.enrollment_id)                    AS enrollments,
  ROUND(100.0 * SUM(CASE WHEN e.status = 'Completed' THEN 1 ELSE 0 END)
        / COUNT(e.enrollment_id), 1)        AS completion_rate_pct,
  ROUND(SUM(c.price_usd), 0)                 AS gross_revenue_usd
FROM courses c
JOIN enrollments e ON e.course_id = c.course_id
GROUP BY c.category
ORDER BY enrollments DESC;
```

RESULT

KEY INSIGHT
Data & Analytics leads with 77 enrollments across 3 courses and \$5,363 gross revenue. Business has the best completion (70.7%); Programming completes only 52.9%.

Method

- 1. JOIN courses to enrollments so each enrollment row carries its course category and price.
- 2. GROUP BY category collapses those rows into five category rows.
- 3. COUNT(DISTINCT course_id) counts each course once; COUNT(enrollment_id) counts learners.
- 4. Completion % = completed rows / all rows; gross revenue = SUM of price across all enrollments.

Functions explained

COUNT (DISTINCT x)	counts each unique value once
CASE WHEN ... END	turns a condition into 1 or 0
SUM (CASE ...)	counts only the Completed rows
ROUND (x, 0)	whole-dollar revenue
JOIN ... ON	links enrollments to their course

How is enrollment growing month by month, and what is the running total?

SQL QUERY

```
-- 3.4 Monthly enrollment trend with running total

WITH monthly AS (
  SELECT DATE_FORMAT(enrollment_date, '%Y-%m') AS month,
         COUNT(*) AS new_enrollments
  FROM enrollments
  GROUP BY DATE_FORMAT(enrollment_date, '%Y-%m')
)
SELECT
  month,
  new_enrollments,
  SUM(new_enrollments) OVER (ORDER BY month) AS cumulative_enrollments,
  ROUND(100.0 * (new_enrollments - LAG(new_enrollments) OVER (ORDER BY month))
        / LAG(new_enrollments) OVER (ORDER BY month), 1) AS mom_growth_pct
FROM monthly
ORDER BY month;
```

RESULT

month	new_enrollments	cumulative_enrollme...	mom_growth_...
2025-02	1	1	NULL
2025-03	2	3	100.0
2025-04	5	8	150.0
2025-05	6	14	20.0
2025-06	3	17	-50.0
2025-07	10	27	233.3
2025-08	4	31	-60.0
2025-09	15	46	275.0
2025-10	9	55	-40.0
2025-11	14	69	55.6
2025-12	17	86	21.4
2026-01	16	102	-5.9
2026-02	17	119	6.3

KEY INSIGHT

Cumulative enrollments reach 119 by Feb 2026. Growth is spiky: Sep 2025 jumped +275% (15 enrollments) and then fell 40% the following month.

Method

- 1. CTE monthly: DATE_FORMAT turns each date into YYYY-MM, then GROUP BY counts new enrollments.
- 2. The outer query reads the CTE and orders the rows by month.
- 3. SUM(new_enrollments) OVER (ORDER BY month) builds the cumulative running total.
- 4. LAG() fetches last month's count to compute month-over-month growth; the first month is NULL.

Functions explained

WITH ... AS ()	CTE: a named, reusable sub-result
DATE_FORMAT(d, fmt)	formats a date as text, e.g. 2025-09
SUM() OVER (...)	window function: running total
LAG(col) OVER (...)	value from the previous row
NULL	no previous month exists for row 1

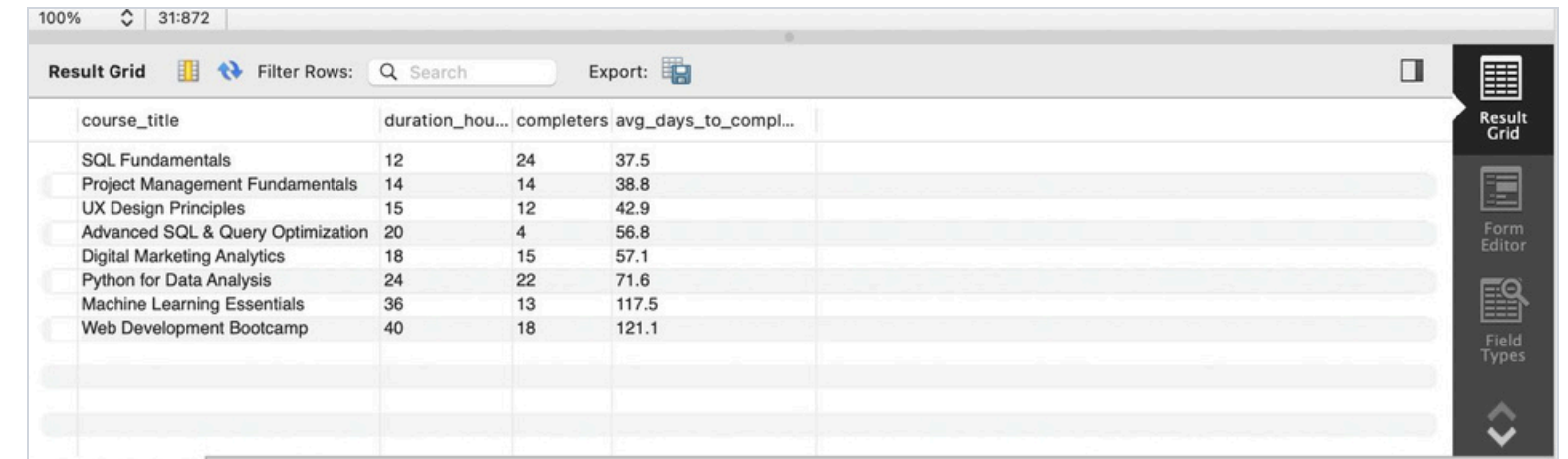
How many days do learners actually need to finish, compared with course hours?

SQL QUERY

-- 3.5 Average time to complete vs. course length

```
SELECT
  c.course_title,
  c.duration_hours,
  COUNT(*)                                AS completers,
  ROUND(AVG(DATEDIFF(e.completion_date, e.enrollment_date)), 1) AS avg_days_to_complete
FROM enrollments e
JOIN courses c ON c.course_id = e.course_id
WHERE e.status = 'Completed'
GROUP BY c.course_id, c.course_title, c.duration_hours
ORDER BY avg_days_to_complete;
```

RESULT



course_title	duration_hou...	completers	avg_days_to_compl...
SQL Fundamentals	12	24	37.5
Project Management Fundamentals	14	14	38.8
UX Design Principles	15	12	42.9
Advanced SQL & Query Optimization	20	4	56.8
Digital Marketing Analytics	18	15	57.1
Python for Data Analysis	24	22	71.6
Machine Learning Essentials	36	13	117.5
Web Development Bootcamp	40	18	121.1

KEY INSIGHT

Time grows with course length, at roughly 3 calendar days per course hour: SQL Fundamentals (12 h) takes 37.5 days, Web Development Bootcamp (40 h) takes 121.1 days.

Method

1. JOIN enrollments to courses to bring in duration_hours.
2. WHERE status = 'Completed' keeps finishers only, the rows that have a completion_date.
3. DATEDIFF(completion_date, enrollment_date) gives days per learner; AVG() averages them per course.
4. ORDER BY avg_days_to_complete lists the fastest courses first.

Functions explained

DATEDIFF (end, start)	number of days between two dates
AVG ()	mean value per group
COUNT (*)	number of completers per course
WHERE	filters rows before grouping
ROUND (x, 1)	1 decimal place

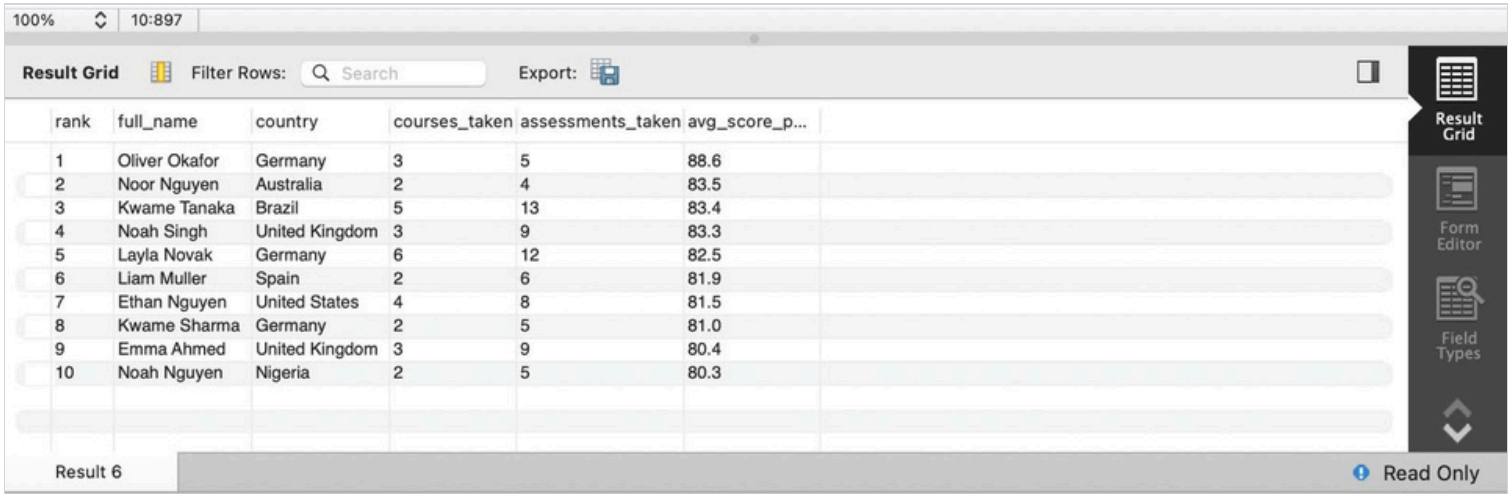
Who are the top 10 learners by average score (minimum 3 assessments)?

SQL QUERY

```
-- 3.6 Learner leaderboard: top 10 by average score (min. 3 assessments)
-- Note: RANK is a reserved word in MySQL 8, so the alias is quoted.

WITH learner_scores AS (
  SELECT
    st.student_id,
    st.full_name,
    st.country,
    COUNT(a.assessment_id)           AS assessments_taken,
    COUNT(DISTINCT e.course_id)     AS courses_taken,
    ROUND(100.0 * SUM(a.score) / SUM(a.max_score), 1) AS avg_score_pct
  FROM students st
  JOIN enrollments e ON e.student_id = st.student_id
  JOIN assessments a ON a.enrollment_id = e.enrollment_id
  GROUP BY st.student_id, st.full_name, st.country
  HAVING COUNT(a.assessment_id) >= 3
)
SELECT
  RANK() OVER (ORDER BY avg_score_pct DESC) AS `rank`,
  full_name, country, courses_taken, assessments_taken, avg_score_pct
FROM learner_scores
ORDER BY `rank`
LIMIT 10;
```

RESULT



rank	full_name	country	courses_taken	assessments_taken	avg_score_p...
1	Oliver Okafor	Germany	3	5	88.6
2	Noor Nguyen	Australia	2	4	83.5
3	Kwame Tanaka	Brazil	5	13	83.4
4	Noah Singh	United Kingdom	3	9	83.3
5	Layla Novak	Germany	6	12	82.5
6	Liam Muller	Spain	2	6	81.9
7	Ethan Nguyen	United States	4	8	81.5
8	Kwame Sharma	Germany	2	5	81.0
9	Emma Ahmed	United Kingdom	3	9	80.4
10	Noah Nguyen	Nigeria	2	5	80.3

KEY INSIGHT
Oliver Okafor (Germany) leads with an 88.6% average across 5 assessments. The top ten range from 88.6% down to 80.3%.

Method

- 1. CTE learner_scores: JOIN students, enrollments and assessments, then GROUP BY student.
- 2. Compute assessments_taken, courses_taken (COUNT DISTINCT) and the weighted average score.
- 3. HAVING COUNT(assessment_id) >= 3 removes learners with too little data (filters after grouping).
- 4. RANK() OVER (ORDER BY avg_score_pct DESC) ranks them; LIMIT 10 keeps the top ten.

Functions explained

HAVING	filters groups after aggregation
RANK () OVER (...)	rank number; ties share a rank
`rank`	backticks: RANK is reserved in MySQL 8
COUNT (DISTINCT ..)	unique courses per learner
LIMIT 10	returns only the first ten rows

3.7

Best-performing course per category

ROW_NUMBER() + PARTITION BY

Which single course has the highest completion rate inside each category?

SQL QUERY

-- 3.7 Best-performing course per category (window function)

```
WITH ranked AS (  
    SELECT  
        category, course_title, completion_rate_pct, total_enrollments,  
        ROW_NUMBER() OVER (PARTITION BY category ORDER BY completion_rate_pct DESC) AS rn  
    FROM course_scorecard  
)  
SELECT category, course_title, completion_rate_pct, total_enrollments  
FROM ranked  
WHERE rn = 1  
ORDER BY completion_rate_pct DESC;
```

Method

1. CTE ranked reads the course_scorecard summary.
2. ROW_NUMBER() OVER (PARTITION BY category ORDER BY completion_rate_pct DESC) restarts numbering inside each category.
3. The outer query keeps only rn = 1, the top course of every category.
4. The CTE is required because a window function cannot be filtered directly in WHERE.

RESULT

category	course_title	completion_rate_pct	total_enrollments
Data & Analytics	SQL Fundamentals	75.0	32
Business	Project Management Fundamentals	73.7	19
Design	UX Design Principles	70.6	17
Data Science	Machine Learning Essentials	61.9	21
Programming	Web Development Bootcamp	52.9	34

KEY INSIGHT

SQL Fundamentals (75.0%) is the top course overall. Web Development Bootcamp is the only Programming course, so its 52.9% is also that category's "best".

Functions explained

ROW_NUMBER()	unique sequence number per row
PARTITION BY category	restarts the numbering per group
ORDER BY ... DESC	inside OVER: best row gets 1
WITH ranked AS ()	CTE that holds the numbered rows
WHERE rn = 1	keeps the winner of each category

How are final-exam results spread across grade bands?

SQL QUERY

```
-- 3.8 Final-exam score distribution

SELECT
  CASE
    WHEN 100.0 * score / max_score >= 90 THEN 'A (90-100)'
    WHEN 100.0 * score / max_score >= 80 THEN 'B (80-89)'
    WHEN 100.0 * score / max_score >= 70 THEN 'C (70-79)'
    WHEN 100.0 * score / max_score >= 60 THEN 'D (60-69)'
    ELSE 'F (<60)'
  END
  COUNT(*)
  ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1)
FROM assessments
WHERE assessment_type = 'Final Exam'
GROUP BY grade_band
ORDER BY grade_band;
```

RESULT

grade_band	students	pct_of_to...
A (90-100)	8	6.6
B (80-89)	27	22.1
C (70-79)	46	37.7
D (60-69)	33	27.0
F (<60)	8	6.6

KEY INSIGHT
Of 122 final-exam attempts, C is the most common band (37.7%). 28.7% score A or B, while 33.6% land in D or F.

Method

- 1. Filter to assessment_type = 'Final Exam'.
- 2. CASE WHEN converts score / max_score into a percentage and assigns a letter band; the first true branch wins.
- 3. GROUP BY grade_band counts final exams per band.
- 4. SUM(COUNT(*)) OVER () adds the grand total to every row, giving each band's share of all exams.

Functions explained

CASE WHEN ... ELSE	if / else-if logic, evaluated in order
COUNT (*)	exams in each band
SUM(COUNT(*)) OVER ()	grand total across all bands
GROUP BY grade_band	groups by the alias of the CASE
ORDER BY grade_band	A to F, alphabetical

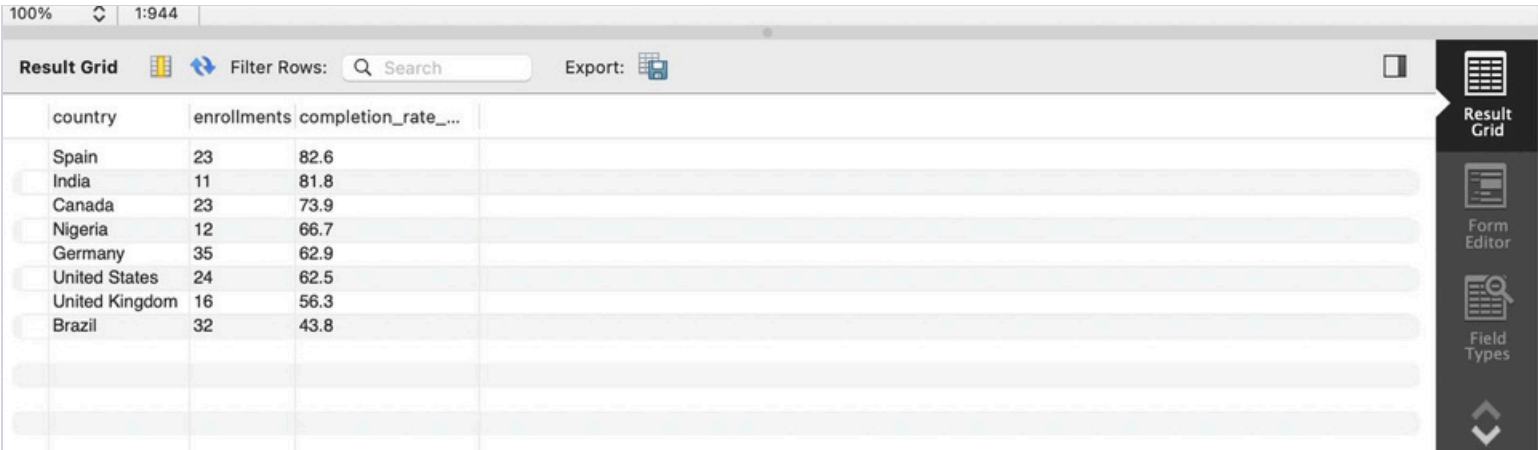
Which countries complete the most courses (at least 10 enrollments)?

SQL QUERY

```
-- 3.9 Completion rate by student country (min. 10 enrollments)

SELECT
  st.country,
  COUNT(*) AS enrollments,
  ROUND(100.0 * SUM(CASE WHEN e.status = 'Completed' THEN 1 ELSE 0 END) / COUNT(*), 1) AS completion_rate_pct
FROM enrollments e
JOIN students st ON st.student_id = e.student_id
GROUP BY st.country
HAVING COUNT(*) >= 10
ORDER BY completion_rate_pct DESC;
```

RESULT



country	enrollments	completion_rate_pct
Spain	23	82.6
India	11	81.8
Canada	23	73.9
Nigeria	12	66.7
Germany	35	62.9
United States	24	62.5
United Kingdom	16	56.3
Brazil	32	43.8

KEY INSIGHT
Spain (82.6%) and India (81.8%) lead. Brazil trails at 43.8% despite 32 enrollments, the largest gap worth investigating.

Method

- 1. JOIN enrollments to students to attach each learner's country.
- 2. GROUP BY country and COUNT(*) the enrollments in each.
- 3. Completion % = SUM(CASE WHEN status = 'Completed' ...) / COUNT(*)
- 4. HAVING COUNT(*) >= 10 drops countries with too few enrollments for a fair comparison.

Functions explained

HAVING COUNT(*) >= 10	minimum-sample filter on each group
CASE WHEN status = ..	flags Completed rows as 1
SUM()	total completed per country
COUNT(*)	all enrollments per country
ORDER BY ... DESC	best completion rate first

Which in-progress learners have gone quiet for 30+ days and are still under 90% done?

SQL QUERY

```
-- 3.10 Stalled learners: in progress, under 90% done, no activity for 30+ days
-- (as-of date fixed at 2026-09-15; use CURDATE() for live data)

SELECT
  st.full_name,
  st.email,
  c.course_title,
  e.enrollment_date,
  e.progress_pct,
  MAX(a.attempt_date) AS last_assessment,
  DATEDIFF('2026-09-15', COALESCE(MAX(a.attempt_date), e.enrollment_date)) AS days_inactive
FROM enrollments e
JOIN students st ON st.student_id = e.student_id
JOIN courses c ON c.course_id = e.course_id
LEFT JOIN assessments a ON a.enrollment_id = e.enrollment_id
WHERE e.status = 'In Progress'
  AND e.progress_pct < 90
GROUP BY e.enrollment_id, st.full_name, st.email, c.course_title,
         e.enrollment_date, e.progress_pct
HAVING days_inactive >= 30
ORDER BY days_inactive DESC;
```

RESULT

full_name	email	course_title	enrollment_da...	progress_p...	last_assessme...	days_inactive
Tara Tanaka	tara.tanaka@example.com	Web Development Bootcamp	2026-04-22	74.2	2026-05-13	125
Kwame Sharma	kwame.sharma@example.com	Web Development Bootcamp	2026-05-06	88.4	2026-05-22	116
Zara Ahmed	zara.ahmed@example.com	Web Development Bootcamp	2026-06-28	84.6	2026-07-22	55
Noor Rossi	noor.rossi@example.com	Web Development Bootcamp	2026-07-12	67.5	2026-08-03	43
Noah Nguyen	noah.nguyen@example.com	Project Management Fundamentals	2026-07-22	81.4	2026-08-06	40
Noor Novak	noor.novak@example.com	Python for Data Analysis	2026-07-20	79.0	2026-08-13	33
Noah Okafor	noah.okafor@example.com	Python for Data Analysis	2026-07-25	69.7	2026-08-13	33

KEY INSIGHT
7 learners are stalled, 4 of them in Web Development Bootcamp. Tara Tanaka has been inactive for 125 days at 74.2% progress: a clear outreach list.

Method

- 1. JOIN enrollments to students and courses for name, email and course; LEFT JOIN assessments.
- 2. WHERE status = 'In Progress' AND progress_pct < 90 keeps unfinished learners.
- 3. Last activity = MAX(attempt_date), or enrollment_date if no assessment exists.
- 4. days_inactive = DATEDIFF(as-of date, last activity); HAVING >= 30, longest inactive first.

Functions explained

MAX(attempt_date)	latest assessment per enrollment
COALESCE(a, b)	first non-NULL value (fallback date)
DATEDIFF(d1, d2)	days between the as-of date and activity
LEFT JOIN	keeps learners with no assessments
CURDATE()	live date; here fixed at 2026-09-15

SQL functions used: quick reference

Function	What it does	Used in report
<code>JOIN / LEFT JOIN</code>	JOIN keeps matching rows only; LEFT JOIN also keeps unmatched rows from the left table	3.2 - 3.10
<code>GROUP BY</code>	Collapses rows into groups so aggregates run once per group	3.2 - 3.10
<code>COUNT()</code> / <code>COUNT(DISTINCT)</code>	Counts rows; DISTINCT counts each unique value once	3.1, 3.3 - 3.6, 3.8, 3.9
<code>SUM()</code> / <code>AVG()</code> / <code>MAX()</code>	Total, average and latest value per group	3.1 - 3.3, 3.5, 3.6, 3.9, 3.10
<code>ROUND(x, n)</code>	Rounds to n decimals for readable percentages	3.1 - 3.9
<code>CASE WHEN ... END</code>	If / else logic inside SQL: flags rows or builds buckets	3.3, 3.8, 3.9
<code>HAVING</code>	Filters groups after aggregation (WHERE filters rows before)	3.6, 3.9, 3.10
<code>DATE_FORMAT()</code> / <code>DATEDIFF()</code> / <code>CURDATE()</code>	Formats dates, counts days between two dates, returns today	3.4, 3.5, 3.10
<code>COALESCE(a, b)</code>	Returns the first value that is not NULL	3.10
<code>WITH ... AS (CTE)</code>	Named temporary result that keeps queries readable	3.4, 3.6, 3.7
<code>SUM()</code> <code>OVER</code> / <code>LAG()</code>	Window functions: running total and previous-row value	3.4, 3.8
<code>RANK()</code> / <code>ROW_NUMBER()</code> <code>OVER</code>	Ranks rows overall or inside each PARTITION BY group	3.6, 3.7

What the reports tell us

33.3%

Advanced SQL completion, the lowest of all eight courses (3.2)

82.6%

Spain's completion rate, the highest of the qualifying countries (3.9)

7

learners stalled 30+ days, ready for outreach (3.10)

Window functions, CTEs, conditional aggregation and date arithmetic turn raw learner data into decisions.

04

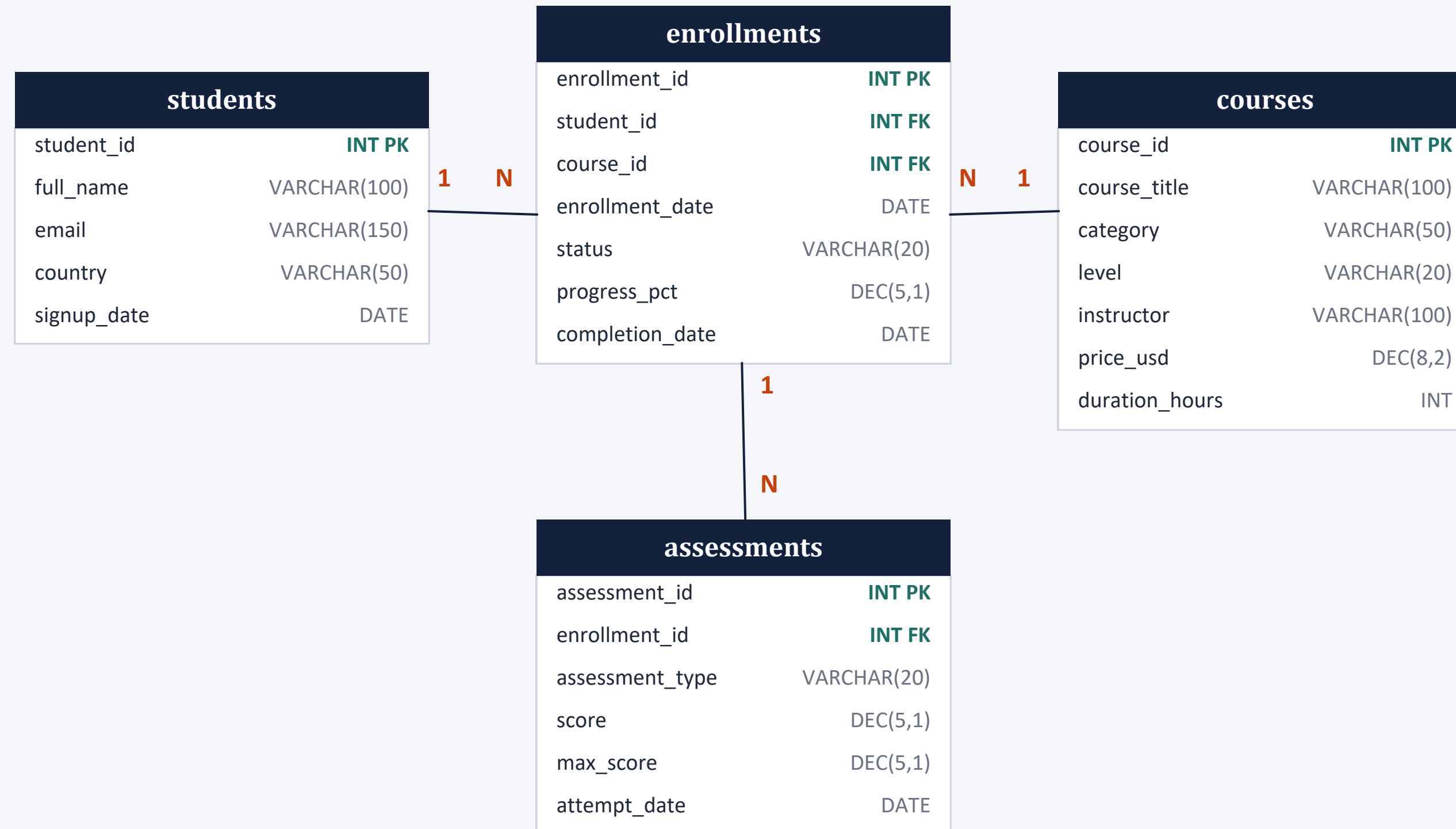
ERD Representation

How the four tables connect through primary and foreign keys

4.1

Entity Relationship Diagram

A visual map of the tables, their columns and how they connect.



1 : N relationships

Relationships

ERD: a visual map of tables, columns and how they connect.

students → **enrollments**: one to many.

courses → **enrollments**: one to many.

enrollments → **assessments**: one to many.

students and courses: many to many, resolved by the enrollments table.

Conclusion

Proper schema design, constraints and SQL aggregation turn raw learner data into clear KPIs: completion, dropout and assessment performance across students, courses and enrollments.

64.2%

completion rate

26.3%

dropout rate

74.6%

average assessment score

Thank you

Online Learning Database | Schema & Reporting